

Forma geral como o projeto está organizado e quais são os arquivos principais e qual é a responsabilidade de cada um.

O projeto segue o padrão MVC simplificado do Flask. Os arquivos principais são três:

- **app.py** — é o coração da aplicação. Contém todas as rotas, a lógica de negócio, o sistema de login, logout e o decorator `login_required` que protege as páginas internas.
- **db.py** — é o módulo de acesso ao banco de dados. Ele cria um pool de conexões MySQL e expõe duas funções principais: `execute_query` para qualquer comando SQL, e `execute_one` para buscar um único registro.
- **schema.sql** — define a estrutura do banco: cria o banco `igreja_membros` e as tabelas `cargos`, `celulas`, `membros` e `contribuicoes` com todos os seus campos e relacionamentos.

Os templates HTML ficam na pasta `templates/`, organizados por módulo — `membros/`, `cargos/`, `celulas/`, `contribuicoes/`. Todos herdam de `base_dashboard.html`, que contém a sidebar, a navbar e as flash messages.

Banco de dados

Foi criado um pool de conexões no `db.py` em vez de abrir uma conexão nova a cada requisição.

Abrir uma conexão com o banco a cada requisição é custoso — envolve autenticação, alocação de recursos e tempo de resposta. O pool de conexões resolve isso mantendo um conjunto de conexões já abertas e prontas para uso. Quando uma requisição chega, ela pega uma conexão disponível do pool; quando termina, devolve essa conexão ao pool em vez de fechar fisicamente.

No projeto usei `MySQLConnectionPool` com `pool_size=5`, ou seja, cinco conexões abertas permanentemente. O método `conn.close()` no bloco `finally` não encerra a conexão de verdade — ele apenas a devolve ao pool para ser reutilizada pela próxima requisição.

Segurança de senhas

As senhas dos usuários são armazenadas no banco e não salvas em texto puro. Salvar senha em texto puro é uma falha grave de segurança — se o banco vazar, todas as senhas ficam expostas imediatamente.

São armazenadas como hash usando a função `generate_password_hash` do Werkzeug. O hash é um texto longo e irreversível — por exemplo, `scrypt:32768:8:1$abc...xyz`. Mesmo que o banco seja comprometido, ninguém consegue recuperar a senha original a partir do hash.

Autenticação e sessão

Como funciona o sistema de login?

O formulário envia um POST para a rota `/login`. O Flask lê o e-mail e a senha com `request.form.get`. Em seguida faz três verificações em sequência:

1. Busca o usuário no banco pelo e-mail com um `SELECT JOIN` que traz também o nome do cargo.
2. Verifica se a senha digitada corresponde ao hash armazenado usando `check_password_hash`.
3. Verifica se o status do membro é `'Ativo'`.

Se tudo passar, os dados do usuário são salvos em `session['usuario']` — um dicionário com id, nome, e-mail, cargo e iniciais. A sessão do Flask usa um cookie assinado pelo `secret_key` para persistir esses dados entre requisições.

No logout, `session.clear()` remove tudo e o usuário volta para o login.

Proteção de rotas

`login_required` é um decorator personalizado que protege as rotas do dashboard. Um decorator é uma função que envolve outra função para adicionar comportamento antes de executá-la.

Quando coloco `@login_required` acima de uma rota, o Flask executa a função `wrapper` antes de executar a rota em si. Dentro do `wrapper`, verifico se `session.get('usuario')` existe. Se não existir, o usuário é redirecionado para o login com uma flash message de aviso. Se existir, a rota é executada normalmente.

Usei `@wraps(f)` do módulo `functools` para preservar o nome original de cada função, porque sem isso todas as rotas decoradas teriam o mesmo nome internamente, causando conflito no Flask.

SQL Injection

Nunca concateno valores diretamente na string SQL. Sempre uso parâmetros separados com `%s` como placeholder e passo os valores em uma tupla separada para o `cursor.execute()`.

- `execute_one('SELECT * FROM membros WHERE email = %s', (email,))`

O `mysql-connector-python` trata esses valores como dados puros, nunca como parte do comando SQL. Então mesmo que o usuário digitasse algo como `' OR '1'='1'`, isso seria tratado como um texto literal, não como código SQL.

Relacionamento entre tabelas

O banco tem quatro tabelas: `cargos`, `celulas`, `membros` e `contribuicoes`.

A tabela `membros` tem dois campos de chave estrangeira: `cargo_id` que referencia `cargos.id_cargo`, e `celula_id` que referencia `celulas.id_celula`. Ambos usam `CONSTRAINT FOREIGN KEY REFERENCES`, o que significa que o banco impede cadastrar um membro com um cargo ou célula que não existam.

A tabela `contribuicoes` tem `membro_id` referenciando `membros.id_membro`. Por isso, ao tentar excluir um membro que tem contribuições registradas, o banco bloqueia a operação com erro de integridade referencial — o que é o comportamento correto para não deixar dados órfãos.

Nos `SELECTs` uso `INNER JOIN` e `LEFT JOIN` para trazer o nome do cargo e da célula junto com os dados do membro, em vez de exibir apenas os IDs.

Templates e herança

O Jinja2 permite que um template filho herde a estrutura de um template pai usando `{% extends %}`. No projeto, `base_dashboard.html` define toda a estrutura da página — o `<head>`, a sidebar, a navbar superior, o bloco de flash messages e os scripts do Bootstrap.

Cada página do dashboard, como `membros/listar.html`, começa com `{% extends "base_dashboard.html" %}` e só precisa preencher os blocos definidos no pai, como `{% block content %}` e `{% block title %}`.

Isso evita repetição de código. Se eu precisar alterar a sidebar ou adicionar um novo script, faço isso em um único lugar — o `base_dashboard.html` — e todas as páginas são atualizadas automaticamente.

Flash messages

Flash messages são mensagens temporárias que o Flask armazena na sessão do usuário para serem exibidas na próxima requisição e depois descartadas automaticamente.

Quando faço `flash('Membro cadastrado com sucesso!', 'success')` antes de um `redirect`, o Flask guarda essa mensagem na sessão. Na próxima página, o template chama `get_flashed_messages(with_categories=True)`, que recupera e remove as mensagens da sessão. O template percorre a lista com `{% for %}` e exibe cada uma como um alerta Bootstrap com a cor correspondente à categoria — `success` em verde, `danger` em vermelho, `warning` em amarelo.

função `garantir_admin`

resolve um problema prático: na primeira vez que o sistema é executado, o banco está vazio e não existe nenhum usuário para fazer login — o sistema ficaria inacessível.

A função é chamada uma vez ao iniciar a aplicação, logo após `iniciar_bd()`. Ela conta quantos membros existem no banco. Se o total for zero, cria automaticamente o cargo "Pastor" e um membro administrador com e-mail `admin@igreja.com` e senha `admin1234` (já armazenada como hash).

Se já existir pelo menos um membro, a função retorna imediatamente sem fazer nada. Isso garante que o sistema sempre tenha um acesso inicial sem precisar de nenhuma configuração manual.

Se acessar diretamente `http://127.0.0.1:5000/membros/listar` sem estar logado

O sistema redireciona para a página de login com a mensagem "Faça login para acessar o sistema." em amarelo.

Isso acontece porque a rota `membros_listar` tem o decorator `@login_required`. Antes de executar a rota, o Flask executa o `wrapper`, que verifica `session.get('usuario')`. Como a sessão está vazia, o `wrapper` chama `flash()` com a mensagem de aviso e retorna um `redirect(url_for('login'))`. A rota original nunca chega a ser executada.